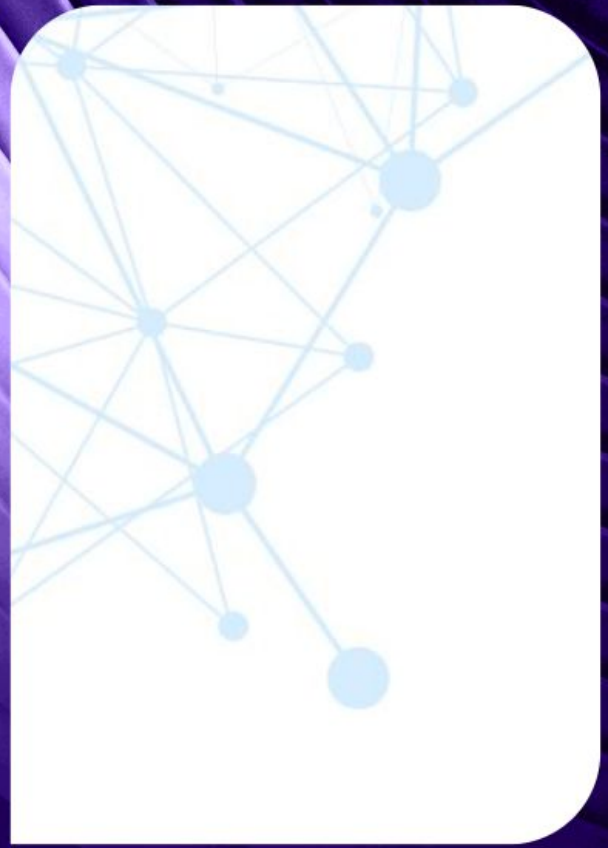
Name: Arushi Varghese
PRN: 25070521001
Year/Semester: Second Year – Third Semester
Institute: Symbiosis Institute of Technology, Nagpur

The Minimum Spanning Tree Problem Using Prim's Algorithm



presentations.ai

Minimum Spanning Tree (MST)

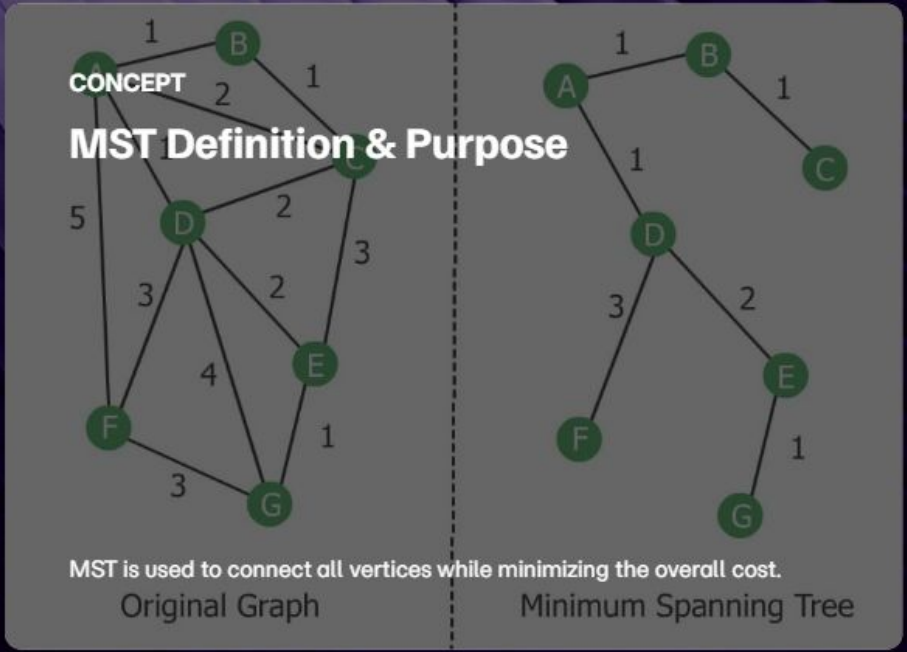
What is a Minimum Spanning Tree?
- A spanning tree is a subgraph of a connected, undirected graph that includes all vertices without forming any cycles.
- A Minimum Spanning Tree (MST) is a spanning tree whose total edge weight is the minimum among all possible spanning trees of the graph.

Key Properties

- Includes all V vertices.
- Contains exactly (V-1) edges, where V is the number of vertices.
- Has no cycles.
- Has the minimum possible total edge weight.

Practical Example

Connecting several cities using roads while minimizing the total construction cost.



presentations.ai

Understanding the Problem Statement

Given a connected, undirected, weighted graph G=(V,E), find a spanning tree with the minimum total edge weight.

The objective is to minimize **$W(T) = \Sigma\, w(u,v)$** where T is the spanning tree and w(u,v) is the edge weight. V = set of vertices and E = set of edges.



Edges and weights:

Why is this important?

Connecting every vertex using arbitrary edges may result in **unnecessary cost**.

Efficiency

An MST identifies the **least expensive way** to connect all vertices without cycles.

presentations.ai

Introduction to Prim's Algorithm

Prim's algorithm is a greedy algorithm used to find the Minimum Spanning Tree of a connected, undirected, weighted graph.

01

Start

Select any vertex as the starting vertex.

02

Initialize

Mark it as part of the MST.

03

Find

Find the minimum-weight edge to an unvisited vertex.

04

Add

Add that edge and vertex to the MST.

05

Repeat

Repeat until all vertices are included.

Why is it called a greedy algorithm? At every step, it chooses the cheapest eligible edge without reconsidering earlier choices. These local choices lead to a globally minimum spanning tree.

presentations.ai

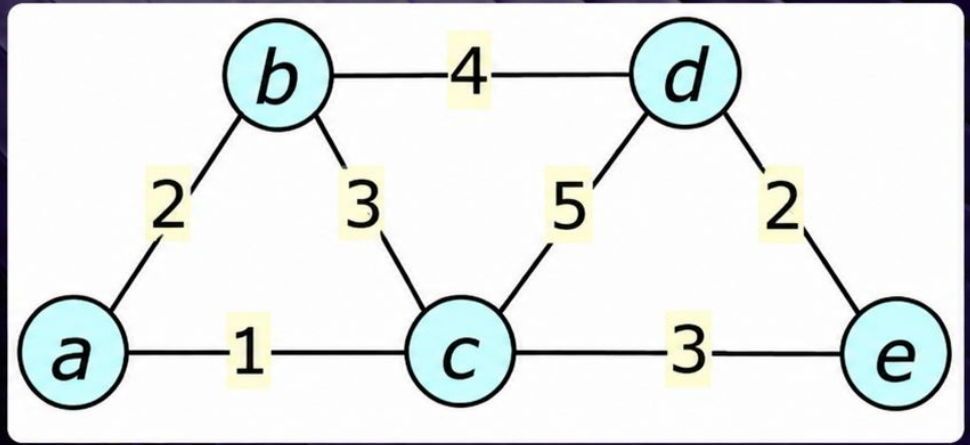
Working of Prim's Algorithm:

Example Graph

Use a weightea graph with five vertices: A, B, C, D, and E.

Edges and weights:

Edge	Weight
A–B	2
A–C	1
B–C	3
B–D	4
C–D	5
C–E	3
D–E	2



presentations.ai

Prim's Algorithm Execution

Start at vertex A, select the minimum-weight edge connecting the tree to an outside vertex.
Goal: Select four edges that connect all five vertices with the minimum total weight.

Step	Current Vertices	Selected Edge	Weight
1	A, B	A–B	2
2	A, B, C	A–C	1
3	A, B, C, D	B–D	4
4	A, B, C, D, E	D–E	2 (Final)

Final MST Edges
The resulting Minimum Spanning Tree connects vertices A–B, A–C, B–D, and D–E.

Total Weight
Summing the weights W(T) = (2 + 3 + 4 + 2) results in a total minimum spanning tree weight of 11.

Observation
The algorithm exclusively selects the minimum weight edges that bridge to an unvisited vertex.

presentations.ai

Algorithm and Pseudocode

The parent array represents the MST edges.

The algorithm performs greedy selection until all vertices are included.

```
PRIM(G, start):
    for each vertex v in G:
        key[v] = INFINITY
        parent[v] = NIL
        visited[v] = false

    key[start] = 0

    repeat V times:
        u = unvisited vertex with minimum key[u]

        if key[u] == INFINITY:
            stop  // graph is disconnected

        visited[u] = true

        for each neighbor v of u:
            if visited[v] == false
               and weight(u, v) < key[v]:
                key[v] = weight(u, v)
                parent[v] = u

    return parent
```

01

key[v]

Stores the minimum edge weight known to connect vertex v to the MST.

02

parent[v]

Tracks the vertex through which v successfully joins the growing MST.

03

visited[v]

Boolean indicator flagging whether a vertex has already been included.

presentations.ai

Time Complexity and Space Complexity

Prim's algorithm can be implemented using different data structures.

Edges and weights:

Implementation	Time Complexity	Suitable for
Adjacency matrix	O(V^2)	Dense graphs
Adjacency list + binary min-heap	O((V+E)log V)	Sparse graphs
Adjacency list + Fibonacci heap	O(E+Vlog V)	Advanced implementations

01 **Efficiency Factors**

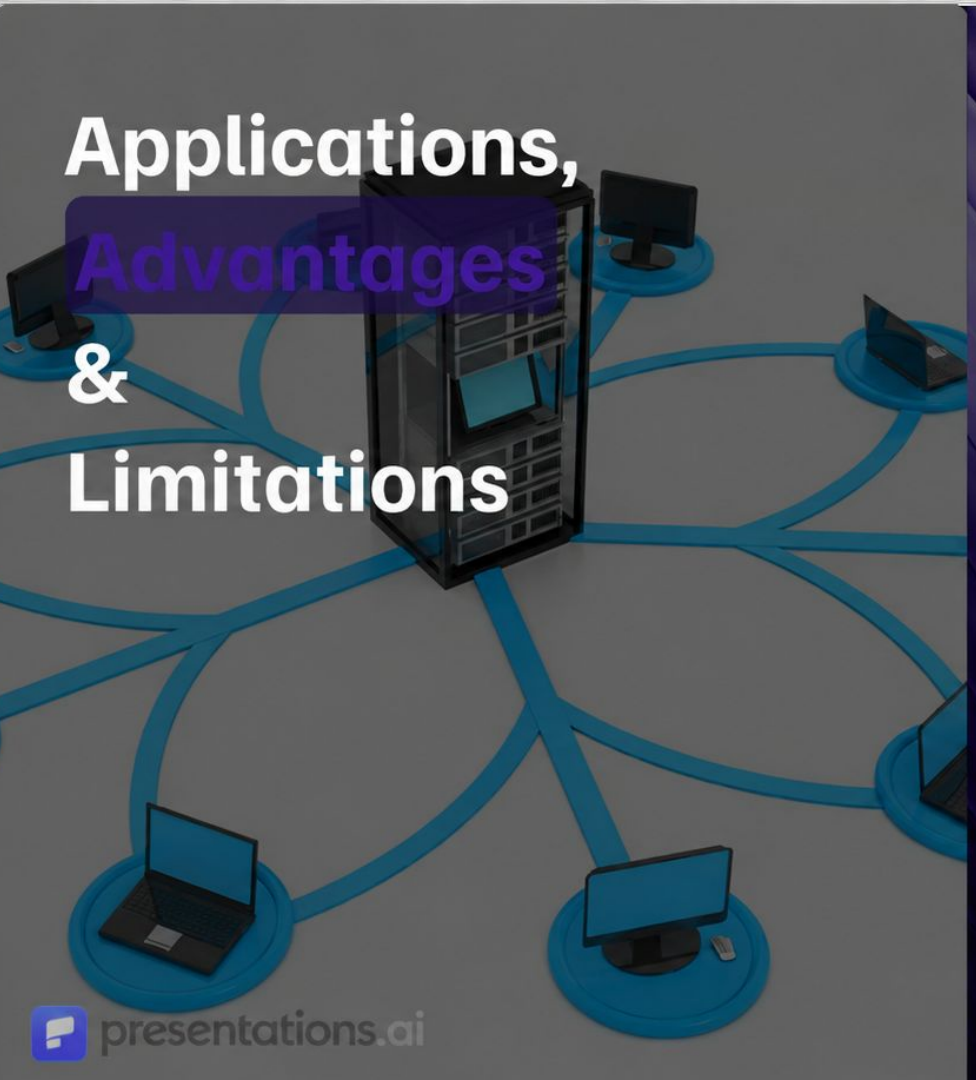
Performance depends on the number of vertices and edges, graph density, and the data structure used to select the minimum-weight edge.

02 **Space Complexity**

Typically O(V+E) complexity when using an adjacency list combined with auxiliary arrays for tracking.

presentations.ai

Applications, Advantages & Limitations



01. Real-world applications

- Designing low-cost comms networks
- Connecting cities with minimal roads
- Planning power distribution grids
- Designing water/cable pipelines

02. Advantages

- Produces min-cost spanning tree
- Simple, systematic greedy approach
- Efficient with proper data structs
- Avoids cycles in all vertices

03. Limitations

- Assumes undirected, weighted graph
- Adapt for disconnected graphs
- Consistent undirected handling
- MSTs may vary with equal weights

presentations.ai

Summary and Conclusion

01

MST Fundamentals

Connects every vertex at the lowest possible total edge weight.

02

Greedy Strategy

Repeatedly selects the cheapest edge connecting to an unvisited vertex.

03

Structure

Contains exactly V-1 edges and no cycles.

04

Efficiency

Performance depends on implementation (matrix vs. min-heap).

Prim's algorithm effectively finds the Minimum Spanning Tree. By selecting the best edge at each step, it minimizes total cost, depending on the graph representation and data structures used.

presentations.ai